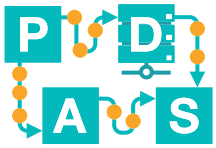


The present work was submitted to:

Chair of Process and Data Science
RWTH Aachen University

Your Thesis Title

Master Thesis

presented by

Johannes Engelhardt
123456

First Examiner: Prof. Dr. Wil M. P. van der Aalst
Second Examiner: Prof. Dr. Second Supervisor

Advisor: Your Advisor

Aachen, November 19, 2025

Acknowledgments

If you want to incorporate it, keep in mind that, according to the official regulations, *independent work* is a very important aspect of writing a thesis. Thus, you can be grateful for the emotional support by your favorite pet, yet we recommend you to be careful if you want to write anything more specific. In general, we suggest to omit this section.

Abstract

The abstract should be a concise summary of what you have done. It should cover, in roughly 1 to 2 sentences per topic:

- The domain, i.e., explain the application domain in which your thesis is applicable.
- The problem, i.e., motivate what the problem is you are solving.
- The method, i.e., describe the method you have proposed, and, compare it, briefly & high level to other approaches. In particular, focus on the benefits of your approach versus the other approach(es)
- The evaluation, i.e., how did you evaluate your method, what results did you obtain and what do the results indicate?

Contents

List of Figures

List of Tables

1 Introduction

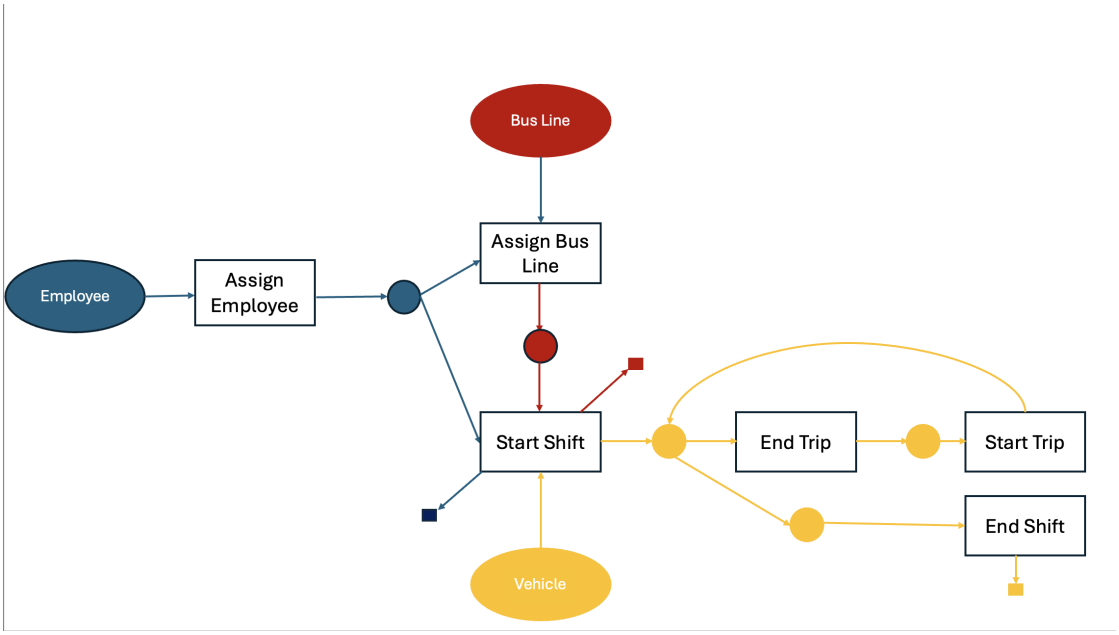
In this section, you introduce the work to your audience. Even though this is the first chapter of your thesis, you should not necessarily write this chapter first. It is often easier to write this chapter at the end of the thesis writing. In this part of your introduction, i.e., the part before the Motivation section (Section 1.1), you write a paragraph (not just a sentence) for each of the bullet points mentioned in the abstract:

- Domain, i.e., explain the application domain in which your thesis is applicable.
- Problem, i.e., motivate what the problem is you are solving.
- Method, i.e., describe the method you have proposed, and, compare it, briefly & high level to other approaches. In particular, focus on the benefits of your approach versus the other approach(es)
- Evaluation, i.e., how did you evaluate your method and what do the results indicate?

Optionally, you can finalize this part with a little overview of what is still to come in this chapter: "The remainder of this chapter is structured as follows. In Section 1.1, we motivate the need for ... In Section 1.2, we provide a formal problem statement. ..."

Before we dive into the motivation section, a few general guidelines for writing:

1. Write *active* and in the *present tense*.
2. Avoid "optionality" as much as possible, i.e., usage of the following words should (pun intended) be minimized:
   - could
   - would
   - should
   - may
   - might
   - can
   - will
   - ...



Good: We present a method that ...
Bad: We will present a method that ...
Good: We use function f and compute its inverse.
Bad: We can use the function f and will compute its inverse.

3. Do not use \\ to generate line breaks, but an empty line
4. Position the caption of a Figure below the figure.
5. Position the caption of a Table above the table.
6. Write Section Headings and Titles Like This
   Good: Approximation Bias in Unstable Systems
   Bad: Approximation bias in unstable systems
7. Be consistent in your references.
   - Make sure that the name of the same author is always the same (using DBLP helps for this)
   - Titles should be consistent. Either use the style previously described, i.e., Approximation Bias in Unstable Systems or Approximation bias in unstable systems (this is allowed in the references, opposed to your own section headings and titles, however, *BE CONSISTENT*).

1.1 Motivation

In this section, you explain to the reader why:

1. The problem you are solving is relevant to be solved.
2. The existing solutions do not solve the problem and/or have significant problems/shortcomings when doing so.

Note that parts of this section are already highlighted in both the abstract and the introduction. However, in this section, you dive a bit deeper. In a good motivation, you show a (simple) example on which current methods fail, yet, the method that you are going to describe in this thesis actually yields a better result.

For example, assume that your thesis describes a new *process discovery* algorithm that is able to handle noise, incomplete behavior, and, on top of that, is able to apply label-splitting. You can take an (example) event log and show that existing algorithms result in models that are of suboptimal quality. Finally, you show a model discovered by your fancy algorithm, and, you explain why this model is so much better.

1.2 Problem Statement

In this section, you introduce the problem that you are solving. A good problem statement is a concise, more general statement of the example motivation that you have used in the motivation section.

For example, in the case of our previous example:

> Real event data contains infrequent and incomplete behavior. Furthermore, different recordings of the same activity may refer to conceptually different contextual executions. Existing, state-of-the-art process discovery algorithms cannot discover process models of adequate quality, given event data of the previously described form.

1.3 Research Questions

The research questions you pose, are questions you need to (largely) answer in order to solve the problem. Basically, the combined set of answers to the questions you pose, allow you to solve the research problem.

From the book of Justin Zobel, "Writing for Computer Science" [**DBLP:books/sp/Zobel14**] (which we highly recommend you to read before writing):

> A hypothesis or research question should be specific and precise, and should be unambiguous; the more loosely a concept is defined, the more easily it will satisfy many needs simultaneously, even when these needs are contradictory. And it is important to state what is not being proposed—what the limits on the conclusions will be.

In the context of our example, we can define many relevant research questions:

1. What are typical prominent noise patterns in real event data?

2. How to detect noise patterns in event data?
3. How to detect infrequent behavior in event data?
4. How to detect contextually different executions of the same activity?
5. How to balance between detected noise patterns, infrequent behavior and imprecise labels?
6. ...

Typically, your thesis consists of 3-5 research questions (often multiple more questions can be defined).

1.4 Research Goals

In this section, you present your research goals. A research goal is a concise statement that s.t., achieving the goal helps you in (partially) solve a research question. Hence, the research questions and goals are often very related. Furthermore, it should be obvious for the reader what goal answers what problem.

In the context of our previously mentioned questions, some example goals are:

- Conduct a systematic literature review on noise patterns in real event data.
- Design a noise detection algorithm.
- Formalize the notion of incompleteness in an event log.
- ...

1.5 Contributions

In this section, you list the contributions that your thesis makes to our wonderful world (of science). Again, there is a strong link to the previous section. Usually, you have achieved your research goals. Hence, the contributions are concrete statements of the goals you have achieved. Additionally, your evaluation (most likely also stressed as a goal) is a contribution. Any implementation or prototype can also be quantified as a contribution.

Some examples:

- A systematic literature review covering 35 articles on noise patterns in real event data
- A noise detection algorithm based on dynamic programming and symbolic linking
- ...

1.6 Thesis Structure

In this section, you outline the remainder of the thesis.

The remainder of this thesis is structured as follows. In Chapter 2, we discuss related work. In Chapter 3, we present basic mathematical preliminaries, used throughout the thesis.

2 Related Work

2.1 Carbon Accounting

2.2 Process Mining for Sustainability

2.3 Object-Centric Process Mining

2.4 Case Notions in Process Mining

3 Preliminaries

3.1 Mathematical Foundations

- An *undirected graph* is defined as a pair $G = (V, E)$ with V being the set of vertices and $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ the set of edges.
  - Two nodes $u, v \in V$ are called *connected* in G whenever $\{u, v\} \in E$.
  - A *path* of length $n \in \mathbb{N}_0$ is a sequence of vertices $\langle v_0, \ldots, v_n \rangle$ with $\{v_i, v_{i+1}\} \in E$ for all $0 \leq i < n$.
  - A vertex $v \in V$ is said to be *reachable* from $u \in V$ if there exists some path in G that starts at u and ends at v.
  - A *connected component* of G is a subset $V' \subseteq V$ where every pair of vertices is reachable from each other, and which cannot be extended by adding more vertices. The collection of connected components of G forms a partition of V and is written as $C(G)$.
  - The *distance* between two vertices $u, v \in V$ is the length n of the shortest path between them, denoted $\text{dist}_G(u, v) = n$. If no such path exists, then $\text{dist}_G(u, v) = \infty$. Formally, $\text{dist}_G : V \times V \to \mathbb{N}_0 \cup \{\infty\}$ with $\text{dist}_G(u, v) = 0$ exactly when $u = v$.
  - The *neighborhood* of a vertex v consists of all vertices directly connected to v. Its size is the *degree* of v, denoted as $\deg_G(v) = |\{u \in V \mid \{u, v\} \in E(G)\}|$.
  - A *clique* in G is a subset of vertices in which every vertex is connected to every other one.

3.2 Object-Centric Event Log

The following universes are defined as pairwise disjoint sets:

- $\mathbb{U}_{ev}$ is the universe of events,
- $\mathbb{U}_{etype}$ is the universe of event types,
- $\mathbb{U}_{obj}$ is the universe of objects,
- $\mathbb{U}_{otype}$ is the universe of object types,
- $\mathbb{U}_{attr}$ is the universe of attribute names,

- $\mathbb{U}_{qual}$ is the universe of qualifiers.

Additionally,

- $\mathbb{U}_{val}$ is the universe of attribute values,
- $\mathbb{U}_{time}$ is the universe of timestamps, including $0, \infty \in \mathbb{U}_{time}$, totally ordered with $0 \leq t \leq \infty$ for all $t \in \mathbb{U}_{time}$.

Definition 3.2.1 (An Object-Centric Event Log (OCEL))**.** An OCEL is a tuple

$$L = (E, O, EA, OA, etype, time, objtype, eatype, oatype, eaval, oaval, E2O, O2O)$$

where

- $E \subseteq \mathbb{U}_{ev}$ is the set of events,
- $O \subseteq \mathbb{U}_{obj}$ is the set of objects,
- $etype : E \to \mathbb{U}_{etype}$ assigns event types to events,
- $time : E \to \mathbb{U}_{time}$ assigns timestamps to events,
- $objtype : O \to \mathbb{U}_{otype}$ assigns object types to objects,
- $EA \subseteq \mathbb{U}_{attr}$ is the set of event attributes,
- $OA \subseteq \mathbb{U}_{attr}$ is the set of object attributes,
- $eatype : EA \to \mathbb{U}_{etype}$ assigns event types to event attributes,
- $oatype : OA \to \mathbb{U}_{otype}$ assigns object types to object attributes,
- $eaval : (E \times EA) \nrightarrow \mathbb{U}_{val}$ assigns values to event attributes,
- $oaval : (O \times OA \times \mathbb{U}_{time}) \nrightarrow \mathbb{U}_{val}$ assigns values to object attributes,
- $E2O \subseteq E \times \mathbb{U}_{qual} \times O$ are the qualified event-to-object relations,
- $O2O \subseteq O \times \mathbb{U}_{qual} \times O$ are the qualified object-to-object relations.

Definition 3.2.2 (Log Graph)**.** A *log graph* $L_G = (E, O, A)$ consists of:

- *event nodes* $E = \{\bullet_i \mid 1 \leq i \leq n\}$,
- *object nodes* $O = \{\bullet_o \mid o \in \Theta\}$,
- *edges* $A = \{(\bullet_i, \bullet_o) \mid o \in O_i\}$,

where event nodes and object nodes are connected by undirected edges.

Table 3.1: An OCEL from a bus public bus transportation process represented by three tables. Events, Objects, and Event-to-Object relations.

Event	Activity	Distance	Timestamp
assign1_e1	assign employee	—	01-09 08:00
start1_v1	start trip	50.00	01-09 08:15
arrive1_v1	arrive stop	55.00	01-09 08:25
validate1_p1	ticket validation	—	01-09 08:26
depart1_v1	depart stop	60.00	01-09 08:34
request1_p1	stop request	—	01-09 08:37
arrrive2_v1	arrive stop	65.00	01-09 08:46
depart2_v1	depart stop	65.00	01-09 08:47
end1_v1	end trip	70.00	01-09 09:00
assign2_e1	assign employee	—	01-09 09:05
start2_v2	start strip	20.00	01-09 09:10
arrive3_v2	arrive stop	30.00	01-09 09:20
depart3_v2	depart stop	30.00	01-09 09:21
end2_v2	end trip	35.00	01-09 09:30

Event	Object
assign1_e1	e1
start1_v1	e1
start1_v1	v1
arrive1_v1	v1
validate1_p1	p1
validate1_p1	v1
depart1_v1	e1
depart1_v1	v1
request1_v1	p1
arrive2_v1	v1
depart2_v1	e1
depart2_v1	v1
end1_v1	v1
assign2_e1	e1
start2_v2	e1
start2_v2	v2
arrive3_v2	v2
depart3_v2	e1
depart3_v2	v2
end2_v2	v2

Object	Obj. type	Consumption
e1	Employee	—
p1	Passenger	—
v1	Vehicle	10
v2	Vehicle	12.5

Definition 3.2.3 (Path in a Log Graph)**.** Let $LG = (E, O, A)$ be a bipartite log graph. A *path* in LG is a sequence

$$\pi = \langle v_1, v_2, \ldots, v_n \rangle, \quad n \geq 2,$$

with $v_i \in E \cup O$ such that $(v_i, v_{i+1}) \in A$ for all $1 \leq i < n$.

We denote by

$$u \rightsquigarrow v$$

that there exists a path π from node u to node v in LG. The notation $u \rightsquigarrow v$ therefore expresses the *reachability relation* between two nodes in the log graph, and will be used throughout this work to indicate that v is reachable from u.

3.2.1 Example Event Log

Table 3.1 presents an example of an object-centric event log (OCEL) describing a bus transportation process. In this setting, the transportation company assigns a driver to a shift. The driver starts the shift with a specific vehicle and subsequently departs

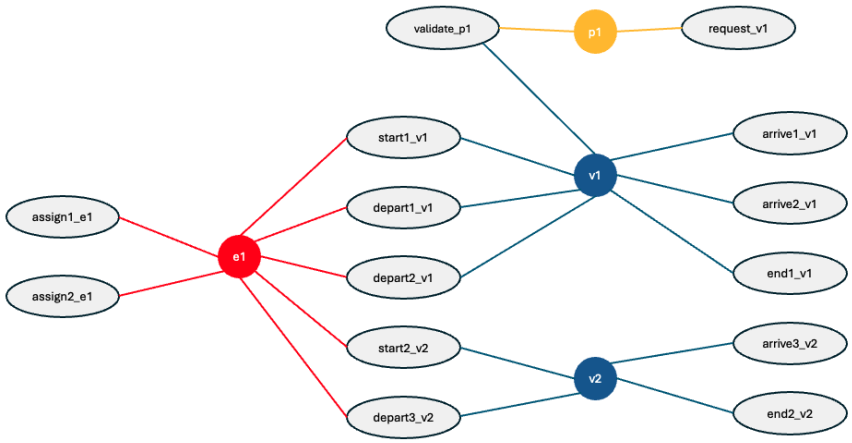


Figure 3.1: Example of a log graph constructed from the event log shown in Table 3.1. Event nodes are shown as white ovals, while object nodes are highlighted in color (e.g., vehicle objects in blue, process object in yellow). The structure illustrates an event–object–event connection: events are indirectly related to each other through shared objects. Starting from the event `depart1_v1`, BFS can traverse the graph via the vehicle `v1` and systematically discover candidate end events.

from and arrives at various bus stations. Along the route, passengers may validate their tickets or request a stop for the next station.

Following the OCEL structure, the data are represented by three tables: *events*, *objects*, and *event-to-object (E2O) relations*. The events (top left table) are represented by unique identifiers and are assigned to an activity, a timestamp, and additional attributes. For example, the activities *depart* and *arrive* contain attributes such as the distance at departure or arrival. The objects (bottom left table) are identified by an ID and an object type, where vehicles additionally carry the attribute *consumption*. The E2O relations (right table) link events with the objects involved in them, thereby forming the core of the object-centric representation.

The example is represented following the OCEL 2.0 standard, which extends OCEL 1.0 by providing a more expressive and standardized representation of events, objects, and their relations.

This minimal example serves as a running example for this thesis. In Chapter 4, we apply our proposed method to this OCEL in order to illustrate the different components of the approach.

3.2.2 Log Graph

Log graphs are a suitable representation of object-centric event data, because they explicitly capture which events are connected to which objects. This structure allows us to determine paths that start in a given event node, end in another event node, and traverse the objects linking them. Such path explorations will be essential in later parts of this thesis, where we analyze relations between start and end events.

Alternative Notation of Object-Centric Event Logs Given an object-centric event log $L = (E, O, \pi_{act}, \pi_{time}, \pi_{omap}, \pi_{vmap})$, we may also use the compact notation

$$L = \langle (a_1, O_1), \ldots, (a_n, O_n) \rangle,$$

where each tuple (a_i, O_i) corresponds to an activity a_i and the set of related objects $O_i \subseteq O$. In this notation, only the relation between events and objects is considered. Other dimensions of the OCEL, such as timestamps or additional attributes, are intentionally ommited at this stage, since the play no role in the construction of the log graph but can be reintroduced in later analysis.

Definition 3.2.4 (Log Graph)**.** Let $L = \langle (a_1, O_1), \ldots, (a_n, O_n) \rangle$ be an object-centric event log. A *log graph* is defined as $LG = (E, O, A)$ with:

- $E = \{\bullet_i \mid 1 \leq i \leq n\}$ the set of *event nodes*,
- $O = \{\bullet_o \mid o \in \Theta\}$ the set of *object nodes*,
- $A = \{(\bullet_i, \bullet_o) \mid o \in O_i\}$ the set of *edges*, each connecting an event node with the objects it refers to.

3.2.3 Handlung Units & Ressource Objects

3.3 Breadth-First Search

Once the log graph has been defined, we require a traversal technique to systematically explore the relations between events and objects. For this purpose, we use the well-known *breadth-first search (BFS)*. BFS explores a graph layer by layer, starting from a given node and first visiting all directly connected nodes before moving on to nodes at a greater distance. This guarantees that the first time a node is reached, the path found is of minimal length with respect to the number of edges.

Algorithm 1: Breadth-First Search (BFS)

Input: Graph $G = (V, E)$, start node $s \in V$
Output: Set of reachable nodes and minimal distances from s
Create a queue Q and initialize with s;
Mark s as visited with distance 0;
while $Q \neq \emptyset$ **do**
    remove first node v from Q;
    foreach *neighbor* u *of* v **do**
        if u *not visited* **then**
            mark u visited with distance $d(v) + 1$;
            add u to Q;

Figure 3.1 shows an example of a log graph. If we select the event *depart1_v1* as a starting node, BFS first discovers the directly connected object *v1*. From there, further events such as *arrive1_v1* or *arrive2_v1* become reachable. In this way, BFS systematically explores all events connected via the same object and thereby collects potential end-event candidates for later analysis.

4 Method

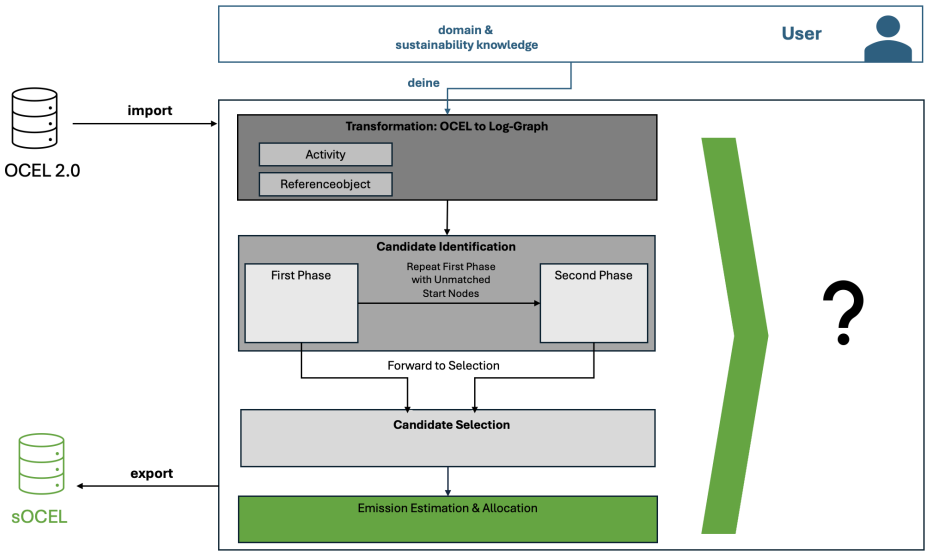


Figure 4.1: Schematic overview of the proposed method. The OCEL 2.0 log is transformed into a log graph, candidate pairs are identified in two phases, selected using a scoring function, and enriched with emissions data to produce the sOCEL output.

The overall goal of our work is to develop a tool that enables emission analysis based on data contained in event logs. We go beyond scenarios where trips are already explicitly represented in the data. In some event logs, driving activities are directly given as specific events. However, in many real cases this is not the situation. Here, trips are not explicitly available but have to be derived from other activities, namely from a start event and a corresponding end event. Only by combining such start and end activities the actual trips can be formed.

Figure 4.1 illustartes how our method is designed to achieve this. After importing the OCEL, the procedure is divied into four phases.

In the first phase, the imported OCEL is transformed into a log graph. The log graph consists of event nodes and object nodes that are connected if an event is related to a certain object. As additional input from the user, the log graph is enriched with infor-

mation about which activities are considered as start events for trips, which activities are considered as end events, and which object type should act as the reference object. This reference object will later serve to connect start and end activities into a trip.

In the second phase, a modified breadth-first search (BFS) is applied on the log graph. The search starts from the user-defined start activities and explores possible end activities that could form a trip together with them. We distinguish between two levels.

- **Phase 2.1** follows the idea of the advanced case notion. Here, resource objects are treated as a stopping criterion, meaning that a path is not continued once such an object is reached. The only exception is the chosen reference object, which must be included in the path so that the resulting trip can be referenced to it.
- **Phase 2.2** is used for the start events where no end candidate could be found in Phase 2.1. In this step the restriction is relaxed and the BFS is also allowed to pass through other resource objects besides the reference object. This way additional candidate pairs of start and end activities can be collected.

In the third phase, the candidate pairs that were collected in the previous step are evaluated. For each possible start–end pair a score is calculated. The score is based on the travel time derived from the timestamps and on how much this duration deviates from an expected travel time provided as expert knowledge. The idea is that the best match should be the one where the observed duration is closest to the expected value.

After calculating the scores, the start–end pair with the lowest score is selected as the most reasonable connection and will be treated as a trip.

In the last phase, the selected trips are enriched with sustainability information. For this step, the attributes that were marked as relevant by the user are used, for example the distance travelled, the vehicle type, or the fuel consumption. Based on these attributes, the corresponding emissions are calculated and assigned to the trip.

The result is a sustainable event log (sOCEL), in which each identified trip is linked to its estimated emissions. This provides the basis for further analysis of the environmental impact of the process.

4.1 Transformation: OCEL to Log-Graph

To leverage the advantages of graph-based representations, the event log needs to be transformed into a graph structure. As we mentioned a log graph in chapter 3 as a graph-based representation of an event log, will be for our purpose not sufficient. Our method needs more information than a standard log graph provides. For example, we require timestamps to check if the duration of a trip is plausible. In addition, we need markings on nodes, such as whether an event is the start or end of a trip, or whether an object is a reference object. For this reason, we define an enriched log graph

Definition 4.1.1 (Enriched Log Graph)**.** Let $L = \langle (a_1, O_1, t_1), \ldots, (a_n, O_n, t_n) \rangle$ be an object-centric event log with activities $a_i \in \Sigma$, sets of objects $O_i \subseteq O$, and timestamps $t_i \in T$.

The *enriched log graph* is defined as

$$ELG = (E, O, A, \tau, type, ref),$$

where

- $E = \{\bullet_e \mid e \in E\}$ is the set of *event nodes*, derived from the events in L,
- $O = \{\bullet_o \mid o \in O\}$ is the set of *object nodes*, derived from the objects in L,
- $A = \{(\bullet_e, \bullet_o) \mid (e, o) \in E2O\}$ is the set of *edges* connecting events with their related objects,
- $\tau : E \to U_{time}$ assigns a timestamp to each event node (inherited from *time* in the OCEL definition),
- $type : E \to \{start, end, normal\}$ classifies event nodes according to their role in the trip detection procedure,
- $ref : O \to \{0, 1\}$ marks whether an object node is a *reference object* (user input, only defined for resource objects).

Definition 4.1.2 (Path in the Enriched Log Graph)**.** Let $ELG = (E, O, A, \tau, type, ref)$ be an enriched log graph. A path π in ELG is a finite alternating sequence of event and object nodes

$$\pi = \langle a_1, o_1, a_2, o_2, \ldots, a_n \rangle, \quad a_i \in E,\ o_i \in O,$$

such that for all consecutive node pairs (x_i, x_{i+1}) it holds that $(x_i, x_{i+1}) \in A$. The set of all paths in ELG is denoted by $\Pi(ELG)$.

Definition 4.1.3 (Event Path)**.** Given a path $\pi \in \Pi(ELG)$, the corresponding event path π_E is the projection of π onto its event nodes:

$$\pi_E = \langle a_1, a_2, \ldots, a_n \rangle \quad \text{with} \quad a_i \in E.$$

An event path therefore represents a sequence of event nodes that are indirectly connected through object nodes in the enriched log graph.

wie bauen wir den graph, aus welcher Tabelle -¿ E2O + user input
Definition Referenzobject

4.2 Candidate Identification

The goal of this section is to identify all possible trip candidates within the event log that could represent a valid trip. As input, we use the previously constructed enriched

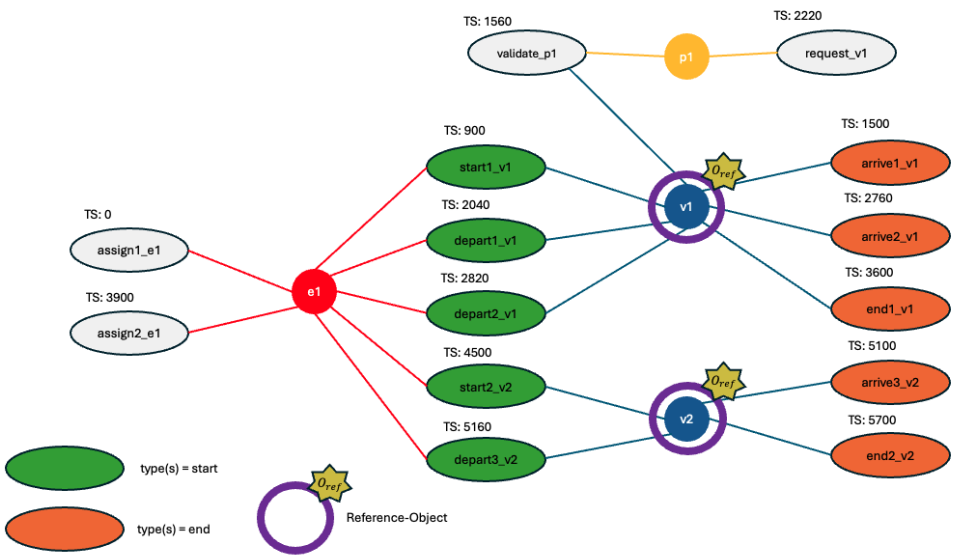


Figure 4.2: Enriched log graph where *start*, *depart* are the assigned start events (green) and *arrive*, *end* are the assigned end events (yellow). In this example, the reference objects is vehicle with its instances *v1*, *v2* with the purple circle. The event nodes hold a simplified version of timestamps where we just see the seconds that passed.

Log Graph, in which events and objects are connected according to their relationships and annotated with timestamps and role information. Each start event serves as the entry point for our search procedure.

To identify potential trip candidates, we perform a traversal of the Log Graph using a modified breadth-first search (BFS). This traversal explores all reachable end events within a predefined temporal and logical boundary, ensuring that only realistic and causally valid connections are considered.

Through this traversal, we obtain for each start event a collection of possible end events that together form the basis for subsequent candidate evaluation and selection.

Definition 4.2.1 (Trip)**.** Let $LG = (E, O, A)$ be a bipartite log graph with $A \subseteq (E \times O) \cup (O \times E)$. A trip is a tuple $f = (s, e, o) \in E \times E \times O_{ref}$ such that

$$
\begin{aligned}
&type(s) = start, \quad type(e) = end,\\
&\exists \pi : s \rightsquigarrow o \rightsquigarrow e,\\
&\Delta t = time(e) - time(s) \in [min\_dur, max\_dur],\\
&\nexists e' \in \pi : type(e') \in \{start, end\},\\
&o = \min_{\pi} \{o' \in O_{ref} \mid o' \in \pi\}.
\end{aligned}
$$

A trip is defined as a tuple consisting of a start event, a reference object, and an end event. The start event must have a path to the reference object, and the reference object

must have a path to the end event. The time difference between start and end event has to be realizable and plausible. No other start or end events are allowed on the path. For the construction of a trip we always take the first encountered reference object along the path.

Consider the example log graph from Figure 4.2. The user assigned *start* and *depart* as events of type start and *arrive* and *end* as events of type end. Furthermore, he defined *v_1*, *v_2* as reference object. For the case, that we consider *start1_v1* as a start node for the modified BFS we get the folling candidates:

(*start1_v1*, v1, *arrive1_v1*), (*start1_v1*, v1, *arrive2_v1*), (*start1_v1*, v1, *end1_v1*)

Definition 4.2.2 (Trip Distance)**.** For a start event $s \in E$, an end event $e \in E$, and an object $o \in O$, the *trip distance* $d(s, e, o)$ is defined as the length of the shortest path in ELG that starts at s, passes through o, and ends at e:

$$d(s,e,o) = \begin{cases} |\pi_{s,e,o}|, & \text{if a path } \pi_{s,e,o} : s \rightsquigarrow o \rightsquigarrow e \text{ exists,} \\ \infty, & \text{otherwise.} \end{cases}$$

4.2.1 Temporal Mode

The Temporal Mode is a mode of candidate selection that aims to identify trips, focusing particularly on the temporal aspect, that is, how events are ordered and spaced in time. We want to develop an algorithm to determine a trip. So far, there is no concrete algorithm, only a definition. Currently, the criterion is that a trip consists of *event nodes* of type *start* and *end*. There must be a path in the log graph in which an activity node of type *start* is followed at some point by an *object node* representing our reference object, and later by an *event node* of type *end*. Additionally, this must occur within a reasonable time frame, and there must be no other start or end nodes on that path.

When using BFS to identify trips, the search initially has no direction or restriction. To introduce structure, we can extend the temporal constraint that already applies to the start and end nodes to all nodes along the path. This means that for any two activity nodes a_i and a_j on the path between start and end, if a_i is discovered before a_j, then $\tau(a_i) < \tau(a_j)$ must hold.

This requirement enforces temporal monotonicity and ensures that the path between start and end follows a consistent and plausible time order. It increases the likelihood that the path represents a real trip, as it excludes connections that are not temporally coherent or causally reasonable.

When using BFS to identify trips, the search initially has no direction or restriction. To introduce structure, we can extend the temporal constraint that already applies to the start and end nodes to all nodes along the path. This means that for any two activity nodes a_i and a_j on the path between start and end, if a_i is discovered before a_j, then $\tau(a_i) < \tau(a_j)$ must hold.

Definition 4.2.3 (Temporal Monotonicity Condition)**.** Let $\pi = \langle a_1, o_1, a_2, \ldots, a_n \rangle$ be a path in the log graph with alternating event and object nodes. π satisfies temporal

monotonicity if for all activity nodes $a_i, a_j \in \pi_E$ with $i < j$ it holds that $\tau(a_i) < \tau(a_j)$.

Corollary 4.2.0.1 (Temporally Valid Trip)**.** *A trip $f = (s, e, o)$ satisfies the temporal monotonicity condition if and only if the connecting path $\pi = \langle a_1 = s, o_1, a_2, o_2, \ldots, a_n = e \rangle$ between s and e is temporally monotone, i.e.,*

$$\tau(a_i) < \tau(a_{i+1}) \quad \forall\, (a_i, a_{i+1}) \in \pi_E.$$

Consequently, the condition implies not only that $\tau(s) < \tau(e)$, but also that the timestamps of all intermediate event nodes along π increase strictly in temporal order.

Consider Figure 4.2 betrachten wir $depart1\_v1$ als startknoten eines trips und die aktivitäten $arrive1\_v1$, $arrive2\_v1$ als endknoten und $vehicle\_1$ als referenzobjekt. Dann ist der pfad $\pi_1 = \langle depart1\_v1, vehicle\_1, arrive1\_v1 \rangle$ kein temporally valid trip, da die temporal monotonicity condition verletzt ist. Für $\pi_1\_E = \langle depart1\_v1, arrive1\_v1 \rangle$ gilt $\tau(arrive1\_v1) < \tau(depart1\_v1)$. Instead der Pfad $\pi_2\_E = \langle depart1\_v1, vehicle\_1, arrive2\_v1 \rangle$ ist ein temporally valid trip, da die temporal monotonicity condition erfüllt ist. Für Für $\pi_2\_E = \langle depart1\_v1, arrive2\_v1 \rangle$ gilt $\tau(depart1\_v1) < \tau(arrive1\_v1)$.

Strict Modified BFS

The goal of this section is to introduce the strict modified breadth-first search (BFS) used for candidate selection. This method serves as the core mechanism for identifying possible trips within the event log. The goal of the strict modified BFS is to identify candidates, i.e. temporally valid trips. As input, we receive the a start event s and the object type and the type of event as a stopping constraint. As output we receive a tuple (s, e, o), consisting of a start event s, and end event e, and a refrence object o. Overall, we receive a selection of candidate tuples.

Our strict modified BFS builds on the idea of the advanced case notion by [**advancedcasenotion**]. Trips are a special form of cases. Similar to their approach, traversal is restricted by resource objects. However, we adapt the concept to trips by starting from an event node instead of an object node and allowing traversal across a chosen reference object. The strict modified BFS proceeds as follows:

- **Start:** The search begins from a designated start event s.
- **Traversal:**
  - Paths may traverse through events and objects.
  - If a vehicle is reached, it is marked as the candidate reference object o.
  - Traversal stops at all other resource objects.
  - Traversal continues only if the timestamp of the next event does not decrease, i.e., for a current event n and a successor event n', it must hold that $\tau(n) \leq \tau(n')$.
- **End:** For each identified reference object o, reachable end events e are collected.

- **Constraints:** Candidate trips must satisfy the time window $[min_d, max_d]$ and the temporal monotonicity condition.

Algorithm 2: Strict BFS

Input: Log graph $G = (E, O, A)$, start events $S \subseteq E$, end events E_{end}, resource type ω_t

Output: Candidate set $C = \{(s, e, o_{res})\}$ and unmatched start events U

Initialize $C \leftarrow \emptyset$;
Initialize $U \leftarrow \emptyset$;
foreach $s \in S$ **do**
  Initialize queue $Q \leftarrow [(s, \tau(s))]$;
  Initialize *visited* $\leftarrow \emptyset$;
  $res\_found \leftarrow False$;
  $end\_found \leftarrow False$;
  $last\_ts \leftarrow \tau(s)$;
  while Q *not empty* **do**
    $(m, last\_ts) \leftarrow Q.pop()$;
    foreach n *neighbor of* m *in* G **do**
      if m *represents an* ***event node*** **then**
        if $n \notin$ *visited* **then**
          Add $(n, \tau(m))$ to Q;
          Add n to *visited*;
      if n *represents an* ***object node*** **then**
        foreach n' *neighbor of* n **do**
          if n' *represents an* ***event node*** **then**
            if $\tau(n') > last\_ts$ **then**
              Add $(n', \tau(n'))$ to Q;
              Add n' to *visited*;
      if m *represents a* ***resource object*** *and* $\omega(m) \neq \omega_t$ **then**
        continue;
      if m *represents a* ***resource object*** *and* $\omega(m) = \omega_t$ **then**
        $res\_found \leftarrow True$;
        $o_{res} \leftarrow m$;
      if res_found ***and*** n *represents an* ***end event*** **then**
        Add (s, n, o_{res}) to C;
        $end\_found \leftarrow True$;
  if $end\_found = False$ **then**
    Add s to U;
return (C, U);

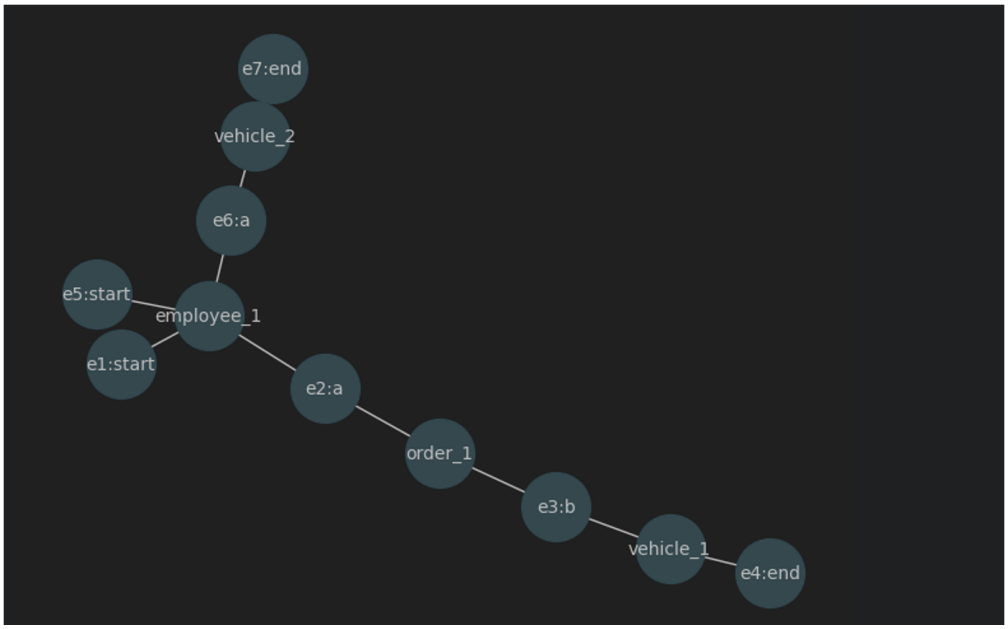


Figure 4.3: Example of a log graph where the path to the reference objects and end events are blocked by an resource object employse

For illustration, consider the start event node $depart1\_v1$ and the reference objects $v1$ and $v2$. In the first step, the strict modified BFS explores its neighbors: the event $e1$ and the object $v1$. Traversal stops at $e1$, since it is a resource object of a different type than the reference object. The object $v1$, however, is the designated reference object, and thus traversal continues.

The neighbors of $v1$ are the events $validate\_p1$ (TS = 1560), $arrive1\_v1$ (TS = 1500), $arrive2\_v1$ (TS = 2760), and $end1\_v1$ (TS = 3600). For $validate\_p1$ and $arrive1\_v1$, the temporal monotonicity constraint is violated, since their timestamps (1560 and 1500) are smaller than the timestamp of the start event $depart1\_v1$ (TS = 2040). Therefore, these nodes are not considered further. The remaining candidates are $arrive2\_v1$ (TS = 2760) and $end1\_v1$ (TS = 3600). Both satisfy the constraint $\tau(depart1\_v1) \leq \tau(arrive2\_v1), \tau(end1\_v1)$ and are marked as potential end events.

As there are no further neighbors to explore, the algorithm outputs the candidate tuples

$$(depart1\_v1, arrive2\_v1, v1), \quad (depart1\_v1, end1\_v1, v1).$$

The strict modified BFS is introduced to avoid complications and ensure that each trip can be identified with exactly one start event, one end event, and one reference object. By enforcing strict traversal rules, the algorithm guarantees that the start event is not accidentally connected to another reference object that does not belong to the same trip. In this way, strict modified BFS prevents ambiguities and avoids competition

between multiple reference objects already at the candidate generation stage.

The example in Figure 4.2.illustrates the advantage: when starting from $depart1_v1$, the algorithm outputs tuples $(depart1\_v1, v1, arrive2\_v1)$ and $(depart1\_v1, v1, end1\_v1)$. In both cases, the reference object $v1$ is clearly determined, ensuring that the trip is consistently defined.

However, strictness can also become a limitation. Consider Figure 4.3 and assume that the traversal starts from $e1 : start$. In this cas point, no connection tose, the first and only neighbor is $employee\_1$, which is a resource object. Since traversal is blocked at thi any end event can be established. This shows that strict modified BFS may fail to identify a trip if resource objects block the path. To handle such situations, we introduce a relaxed variant in the following subsection.

Incremental Relaxed Modified BFS

Relaxed Modified BFS

As mentioned in the previous subsection, the search for candidates can get stuck when the traversal encounters a resource object. To still compute candidates in such cases, we introduce a relaxed variant that allows us to continue the search even if resource objects are present on the path. The goal of this method is again the identification of tuples (s, o, e), but specifically for cases where the strict modified BFS does not return any candidates. The relaxation enables traversal across resource objects that would otherwise block the path in the strict variant. The procedure is similar to the strict BFS, but with the difference that resource objects other than the reference object can also be passed during traversal.

Algorithm 3: Relaxed BFS (Second Pass)

Input: Set of unmatched start events U from strict BFS, log graph $G = (E, O, A)$, resource type ω_t
Output: Additional candidate set $C_{relaxed} = \{(s, e, o_{res})\}$

Initialize $C_{relaxed} \leftarrow \emptyset$;
foreach $s \in U$ **do**
  Initialize queue $Q \leftarrow [(s, \tau(s))]$;
  Initialize $visited \leftarrow \emptyset$;
  $res\_found \leftarrow False$;
  $last\_ts \leftarrow \tau(s)$;
  while Q *not empty* **do**
    $(m, last\_ts) \leftarrow Q.pop()$;
    foreach n *neighbor of* m *in* G **do**
      if m *represents an* ***event node*** **then**
        if $n \notin visited$ **then**
          Add $(n, \tau(m))$ to Q;
          Add n to $visited$;
      if n *represents an* ***object node*** **then**
        foreach n' *neighbor of* n **do**
          if n' *represents an* ***event node*** **then**
            if $\tau(n') > last\_ts$ **then**
              Add $(n', \tau(n'))$ to Q;
              Add n' to $visited$;
      if m *represents a* ***resource object*** *and* $\omega(m) = \omega_t$ **then**
        $res\_found \leftarrow True$;
        $o_{res} \leftarrow m$;
      if m *represents a* ***resource object*** *and* $\omega(m) \neq \omega_t$ **then**
        continue traversal without stopping;
      if res_found ***and*** n *represents an* ***end event*** **then**
        Add (s, n, o_{res}) to $C_{relaxed}$;
return $C_{relaxed}$;

For illustration, considerFigure 4.3. We take the start events $e1 : start$ and $e5 : start$ and the reference object type *vehicle*. Both start events are contained in the set U, since in the previous round the strict BFS did not find any corresponding end events for them, as the object *employee_1* blocked the traversal.

We begin with $e1 : start$. Immediately after the BFS starts, the traversal reaches the resource object *employee_1*. In the strict variant, the search would terminate at this point. However, in the relaxed BFS, the traversal continues. After traversing

employee_1, two branches emerge.

The first branch follows the path $e2 : a \rightarrow order\_1 \rightarrow e3 : b \rightarrow vehicle\_1 \rightarrow e4 : end$. For this branch, the object *vehicle_1* is identified as the reference object, as it is the first reference object encountered along the path. Importantly, the search for potential end events begins only after a reference object has been found. This activation step ensures that the trip is delimited by a start and an end event associated with the same reference object, i.e., the object that conducts or is referenced by the trip.

The second branch consists of the path $e6 : a \rightarrow vehicle\_2 \rightarrow e7 : end$. Since *vehicle_2* is the first reference object encountered in this branch, it is selected as the reference object. Both branches are thus marked as valid candidates.

4.2.2 Atemporal Mode or Relaxed Temporal Mode

Similar to the temporal mode, the relaxed temporal mode also conducts candidate selection and identifies potential trips. However, we soften the temporal monotonicity from the temporal mode and thereby allowing additonally potentially valid trip pairs to be considered. Because the enriched log graph only stores which events interact with which objects and not the full chronological order, the idea of temporal monotonicity quickly runs into its limits.

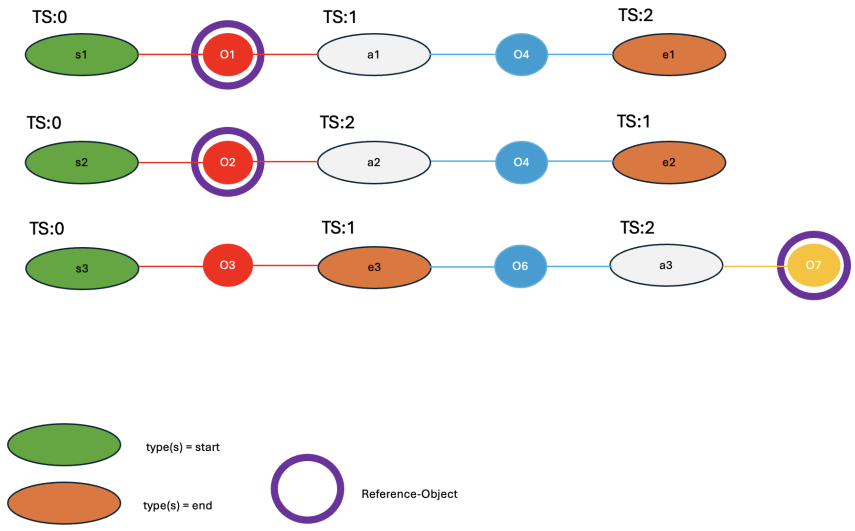


Figure 4.4: Three different Enriched Log Graphs, two different outcomes. The top example, leads to a temporal valid trip as the temporal monotonicity condition is not violated. For the examples below, the temporal monotonicity is violated. The temporal mode has here its limit.

Temporal Conflict on a Path. hier jetzt noch eingeschaften schreiben zu temporal conflicten. einmal die bedeutung davon dass die topologie zusammen mit den timestamps

und zusammen mit den temporal monotonicity constraint dazu führt, dass wir keine paare finden. Dass ausserdem mit der annahme dass wir dachten dass die ref objecte zwischen den start und end event liegen nicht war ist und dass die temporal monotonicity probleme damit macht weil wenn wir um diese zu finden diese monotonicity brechen müssen

Lemma 4.2.1 (Temporal Disorder on Log-Graph Paths)**.** *Let $\pi = \langle a_1, o_1, a_2, o_2, \ldots, a_n \rangle$ be a path in the enriched log graph. Since the log graph encodes object–event relations but no temporal structure, the sequence of timestamps along $\pi_E = \{a_1, a_2, \ldots, a_n\}$ is not constrained to be monotone. In particular, for any pair of indices $i < j$, it is possible that*

$$\tau(a_j) < \tau(a_i).$$

Lemma 4.2.2 (Temporal Independence of the Reference Object)**.** *Let $s, e \in E$ be event nodes with type(s) = start, type(e) = end, and $\tau(s) < \tau(e)$, and let $o \in O$ be a reference object such that (s, e, o) forms a potential trip. Define*

$$E(o) = \{\, a \in E \mid (a, o) \in A \,\},$$

i.e. the set of event nodes connected to o. Then the temporal interval of o,

$$[\min \tau(E(o)),\ \max \tau(E(o))],$$

is independent of its structural position in the log graph. In particular, it may lie before, between, or after the timestamps of s and e.

Corollary 4.2.2.1. *Due to temporal disorder on paths and the temporal independence of the reference object, a structurally valid trip (s, e, o) may fail the temporal monotonicity condition.*

Lets consider Figure 4.4 and assume that the only ressourece objects are $O1, O2, O7$. For the top example, we have an enriched log graph where we can put over a chronological order. We can observe, that we can form a temporal valid trip $(s1, e1, O1)$, where $s1$ is an activity node of type start, $e1$ is an activity node of type end and $O1$ is the reference object for the trip. The next example below shows the limits of the temporal monotonicity approach. Lets assume $(s2, e2, O2)$ are actually a trip. If we conduct the the temporal mode of the previous subsection we have a secquence $\tau(s2) < \tau(a2) \not< \tau(e2)$. As mentioned, the the enriched log graph does not depict the chronological order and with that if an activity uses the objects $O2$ and $O4$ after $s2$,$e2$ it lies in between the the and *blocks* the path such that we can receive the trip. For the last example consider that the actual trip is $(s3, O7, e3)$. The temporal monotonicity and the temporal valid trip implies that the reference object have to lie on the path between the start type activity and the end type activity. Sometimes, the temporal monotonicity prevent building trips as the reference object lies from a time perspective behind the start and end event. This occurs in the lowest example. If we consider the timestamp $a3$, it is greater than the timestamp of $s3$,$e3$. Our modified BFS would first move from

$s3 \to O3 \to s3 \to O6 \to s3 \to O7 \to s3$ and would stop here as $\tau(s3) \not< \tau(e3)$ as the temporal monotonicity condition would be violated. Therefore, we need a another perspective to work on a log graph. First we define

4.2.3 Missing Reference Object Mode

4.3 Candidate Selection

The previous section described how candidates for trips can be identified in the OCEL. In this section, we focus on selecting which of these candidates is most suitable to represent a trip. To this end, we introduce a scoring mechanism that evaluates all identified candidates and selects the one with the highest score. The idea behind this scoring is based on a simple observation: in many domains, such as public transportation or parcel delivery, trips between certain locations usually follow an expected duration. For example, if we consider a bus trip between two stations, the expected travel time may be around one minute. Of course, real trips can deviate from this value, for instance due to traffic jams or delays. However, if we compare two possible trip candidates, one with a duration of three minutes and one with ten minutes, the candidate closer to the expected value is more likely to be the correct one. Very short durations, such as one second, as well as unrealistically long durations, were already addressed in the previous section. What remains is to distinguish between the more plausible candidates. For this reason, we introduce a scoring function that compares the observed trip duration with the expected value. The candidate with the smallest deviation receives the best score and is selected as the final trip.

Definition 4.3.1 (Trip Scoring Function)**.** Let $ELG = (E, O, A, \tau, type, ref)$ be an enriched log graph with $\tau : E \to U_{time}$. The trip scoring function is defined as

$$score : E \times E \times \mathbb{R}_{\geq 0} \longrightarrow \mathbb{R}_{\geq 0},$$

$$score(s, e, \mu) = \left|\tau(e) - \tau(s) - \mu\right|,$$

where

- $s, e \in E$ with $type(s) = start$ and $type(e) = end$,
- $\mu \in \mathbb{R}_{\geq 0}$ is the expected duration.

Definition 4.3.2 (Trip Selection)**.** Let $s \in E$ be a start event with $type(s) = start$. For s, let $\mathcal{C}(s) \subseteq \{s\} \times E \times O$ denote the set of candidate triples (s, e, o) with $type(e) = end$.
The selected trip is defined as

$$(s, e^*, o^*) \in \underset{(s,e,o)\in\mathcal{C}(s)}{\arg\min}\ score(s, e, \mu).$$

As introduced in subsubsection 4.2.1, wie identified candidate trips for the log graph in example Figure 4.2 for the start event $s = depart1\_v1$. According to Definition 4.3.2, the final trip is obtained by choosing the candidate with the minimal score.

When consider the start event $depart1\_v1$, the candidate set is

$$\mathcal{C}(depart1\_v1) = \{(depart1\_v1, arrive2\_v1, v1), (depart1\_v1, end1\_v1, v1)\}$$

With an expected duration of $\mu = 600$ and applying the socring function, we obtain

$$score(s, e_1, \mu) = |\tau(arrive2\_v1) - \tau(depart1\_v1) - 600| = |2760 - 2040 - 600| = 120,$$
$$score(s, e_2, \mu) = |\tau(end1\_v1) - \tau(depart1\_v1) - 600| = |3600 - 2040 - 600| = 960.$$

Applying the selection rule (Definition 4.3.2), we obtain

$$(s, e^*, o^*) \in \arg\min_{(s,e,o)\in\mathcal{C}(depart1\_v1)} score(s, e, \mu),$$
$$(s, e^*, o^*) = (depart1\_v1, arrive2\_v1, v1),$$

In this case (depart1_v1, arrive2_v1, v1) is preferred to (depart1_v1,end1_v1,v1) as we have $score(depart1\_v1, arrive2\_v1, \mu)$=120 compared to $score(depart1\_v1, end1\_v1, \mu)$=960.

While the proposed method provides a systematic way to select trips and ensures robustness when using strict BFS, there are still limitations to be addressed. The scoring is currently one-dimensional, relying only on time, and a single global expected value μ is often insufficient in realistic transportation settings. Rush hours, different city areas, or varying topographies can all affect trip duration and require context-sensitive expected values. Moreover, a tie-break rule is needed when two candidates result in the same score, and in the future, a global matching strategy could further improve consistency across trips. In the following section, we build on these results and introduce the estimation of emissions for the selected trips.

Definition 4.3.3 (Global Trip Set)**.** Let $S_{\text{start}} \subseteq E$ denote the set of all start events with $type(s) = start$. For each $s \in S_{\text{start}}$ and (s, e^*, o^*) the selected trip. We define the *global trip set* T^* as

$$T^* = \{(s, e^*, o^*) \mid s \in S_{\text{start}},\ (s, e^*, o^*) \in \arg\min_{(s,e,o)\in C(s)} score(s, e, \mu)\}.$$

This set contains all selected trips obtained by applying the trip selection rule to every start event in the log.

4.4 Handling Ambiguities in the Matching Process

In the previous section, we proposed a scoring-based approach to select the most suitable end event for each start event. However, as we observed before, multiple candidates can achieve the same score, resulting in ambiguous situations where no clear best match can be determined. In this subsection, we adress how such abmiguities can be handled systematically. We distinguish between two types of ambiguities: *local* and *global*.

- A *local ambiguity* occures, when for one start event, several end event candidates

receive the same best score

- A *global ambiguity* occurs when two differen start events are both matched to the same end event

In this subsection, we analyse these two types o ..... hierfehlt noch was

4.4.1 Local Ambiguities

Definition 4.4.1 (Local Ambiguity)**.** Let $s \in E$ be a start event with $type(s) = start$ and let $\mathcal{C}(s) = \{(s, e, o) \mid e \in E,\, o \in O\}$ denote the set of candidate triples identified for s. Let $score(s, e, \mu)$ be the trip scoring function. Local ambiguity is defined as

$$\left| \arg \min_{(s,e,o) \in \mathcal{C}(s)} score(s, e, \mu) \right| > 1.$$

In such a case, more than one end-event candidate is equally optimal for the same start event s.

Handling Local Ambiguity with First Comes First Serves

In case two or more triplets $(s, e, o) \in C(s)$ have the same score, a *first-come-first-serve* strategy is applied, meaning that the triplet which appears first in the candidate set $C(s)$ is selected.

Handling Local Ambiguity with Distance Score

An alternative way to resolve local ambiguity is to select the trip with the shortest path. If two or more candidate trips for a start event have the same score and their end events are in competition, the *trip distance* $d(s, e, o)$ can be used as a secondary criterion. In this case, the trip with the smaller distance is preferred.

Handling Local Ambiguity with Resource-Based Preference

A third criterion can be derived from the case notion proposed by van Detten et al., which suggests that resource objects tend to connect multiple cases and thereby blur their semantic boundaries. Since a trip is considered a subset of a case, we aim to form trips that involve as few resource objects as possible. Based on this idea, the number of traversed resource objects serves as an additional quality indicator: the fewer resource objects a trip passes through, the more likely it is to represent the correct and semantically coherent trip.

4.4.2 Global Ambiguities

In the previous subsection, we examined *local ambiguities* in the trip selection process. At this stage, each start event was assigned to an end event without considering whether that end event had already been associated with another start event. In other words, the

local assignment did not enforce exclusivity on the end events. Based on the previously defined *global trip set*, which collects all selected trips across the log, we observe that each start event occurs exactly once in the set, whereas the same end event may appear in multiple selected trips. This phenomenon, where distinct start events are linked to the same end event within the global trip set, is referred to as *global ambiguity*, describing the remaining non-uniqueness in the overall event mapping.

In the following subsection, we aim to address this issue by introducing approaches that ensure a one-to-one correspondence between start and end events, such that each end event is matched to exactly one start event and vice versa. To this end, we analyze two types of matching strategies: *Greedy Matching* and *Bipartite Weighted Matching*.

Definition 4.4.2 (Global Ambiguity)**.** Let T^* denote the global trip set containing all selected triplets (s, e, o) with $s \in S_{\text{start}}$, $e \in E$, and $o \in O$. A *global ambiguity* occurs if there exist at least two distinct triplets $(s, e, o), (s', e, o') \in T^*$ with $s \neq s'$ that share the same end event e. Formally, for an end event $e \in E$ with $type(e) = end$, a global ambiguity is present if

$$\left|\{(s, e, o) \in T^* \mid s \in S_{\text{start}},\, o \in O\}\right| > 1.$$

This condition indicates that multiple start events are associated with the same end event within the global trip set, leading to a non-unique global mapping between start and end events.

Greedy Matching

One simple approach to resolving global ambiguity is to sort the global trip set T^* in ascending order by the score value and iteratively select the trips with the lowest scores. Once a trip (s, e, o) is selected, the corresponding end event e is considered blocked and can no longer appear in any other trip within T^*. If, during iteration, another trip containing the same end event e is encountered in a lower rank of the sorted list, a new end event must be recomputed for the corresponding start event s. For the following definition, we assume that the set of trips T^* can be totally ordered. In cases where multiple trips share the same score, we apply additional sorting dimensions introduced in the subsection on local ambiguity. Specifically, we first compare trips by their path length, preferring shorter paths. If ties remain, we consider the number of passed resource objects, giving priority to trips that involve fewer resources. Finally, if a total order still cannot be established, we apply a *first-come-first-serve* strategy, prioritizing the trip whose end event would occur first. This procedure ensures a total order, which is always achievable in technical systems where event sequences are temporally ordered.

Definition 4.4.3 (Greedy Matching)**.** Let T^* denote the global trip set containing all selected triplets (s, e, o) with the associated score function $score(s, e, o) \in \mathbb{R}_{\geq 0}$. We define an ordering relation $\prec$ on T^* such that for any $(s, e, o), (s', e', o') \in T^*$ it holds that

$$(s, e, o) \prec (s', e', o') \iff score(s, e, o) < score(s', e', o').$$

The *Greedy Matching* iteratively constructs a set of final trips $F_T \subseteq T^*$ by traversing T^* in ascending order with respect to $\prec$ and applying the following update rule for each $(s, e, o) \in T^*$:

$$F_T \leftarrow \begin{cases} F_T \cup \{(s, e, o)\}, & \text{if } e \notin E_{\text{used}}, \\ F_T, & \text{otherwise.} \end{cases}$$

After each iteration, the set of used end events is updated as

$$E_{\text{used}} \leftarrow E_{\text{used}} \cup \{e \mid (s, e, o) \in F_T\}.$$

All triplets $(s', e, o') \in C(s')$ with $e \in E_{\text{used}}$ are subsequently removed from their respective candidate sets.

hier beispiel

After the initial greedy matching, we obtain a reduced set of start events S_{start} for which no end event has yet been assigned. Consequently, the corresponding candidate sets $C(s)$ become smaller, as end events that are already assigned can no longer be considered. We then repeat the trip selection step as described in Definition 4.3.2, selecting one new trip for each remaining candidate set, and apply the greedy matching again to the newly selected trips. This process is repeated iteratively, further reducing the sets S_{start} and $C(s)$ in each iteration, until no end events remain available. At this point, the procedure terminates.

Bipartite Matching

4.4.3 Handling Outliers

4.5 Emission Estimation & Allocation: Building the sOCEL

5 Evaluation

In this chapter, you present your evaluation. You should have extensively discussed with your supervisor what you are evaluating and why ...

Some typical sections that you need here:

5.1 Setup

The evaluation consists of two parts. In the first part, we examine whether the strict mode produces better results in terms of the calculated score compared to the relaxed mode. In the second part, we perform an experimental evaluation to analyze for which types of input the algorithm successfully identifies pairs. The experiments are conducted on three OCELs, which are introduced after defining the experimental setup.

For each dataset, we extracted several pairs of activities that can be categorized based on their likelihood of occurrence: likely, moderately likely, and unlikely. For each pair, we applied different object types, specifically resource objects and handling units. In the experimental setup, all tests were conducted under the assumption that each case consists of a start event, followed by a reference object, and finally an end event.

5.2 Results

Show the results. Try to present your results as structured as you can. A good result (sub)section, follows the following structure:

- Describe what results you are going to Show
- Present an initial hypothesis about the results, e.g., We expect the quality metric M to behave like this, conditional to this parameter P.
- Show the Results
- Confirm the hypothesis.
- If there are results that are not according to the hypothesis, you have to be able to explain why this is the case!!!

5.2.1 Strict BFS vs. Relaxed BFS

5.2.2 Epxerimental Evaluation of Trip Formation

5.2.3 Experimental Evaluation of Pair Formation

The goal of the experimental evaluation was to understand how the algorithm behaves under different types of input. In this part, we wanted to see whether the algorithm can still produce results when we provide input data that are not directly related to transportation processes. The main questions were therefore: (1) do we get any results at all when using non-typical input, (2) how do the output values look, and (3) why are no pairs found for some inputs.

To answer these questions, we tested the algorithm on several OCEL event logs and varied the parameters of the input, such as the time intervals and data segments. By changing these settings, we could observe how the algorithm reacts to different time windows and how sensitive it is to missing or incomplete data. In addition, we also wanted to find potential false positives or unexpected results to better understand in which situations the algorithm still produces pairs and when it fails to do so.

Hypothesis. Our main hypothesis is that if there exists a start event s and an end event e that are close to each other in the log graph, the algorithm should always be able to identify them as a valid trip. We define two events s and e as *close* if there exists a path between them in the log graph $LG = (E \cup O, A)$ and if their timestamps satisfy the temporal constraint

$$|\tau(e) - \tau(s)| \leq \Delta t.$$

Here, $\tau : E \to \mathbb{T}$ denotes the timestamp function that assigns each event $e \in E$ its occurrence time, and Δt represents the maximum allowed time interval for considering two events as temporally close. Under this assumption, a valid trip should always be discovered whenever a start and an end event fulfill both the graph connectivity and the temporal proximity condition.

The results of the experiments provide valuable insights into aspects of the algorithm that we have not yet considered. They help us to evaluate whether our initial assumption — that a trip in the graph always follows the order of a start event, a reference object, and an end event — is sufficient for all situations. Based on the findings, it might be necessary to extend this assumption or to define an additional method that follows a different premise. Such an alternative approach could allow the user to choose between different modes depending on the structure of the event log and the intended use case.

Bus Transportation

The bus transportation event log contains three object types: *Order*, *Vehicle*, and *Employee*. The objects *Vehicle* and *Employee* represent resource objects, while *Order* can be understood as a handling unit that links operational activities. The log includes three main activity types: *start*, *a*, and *end*. For the experimental evaluation, we focus on the

activities *start* and *end* as the start and end events of a potential trip, and on the objects *Order* and *Vehicle* as the key entities connecting these events.

During the experiments, certain event–object configurations proved to be particularly interesting. One notable example is the constellation *(end, start, order)*, where the end event occurs before the next start event but is still connected through the same order object.

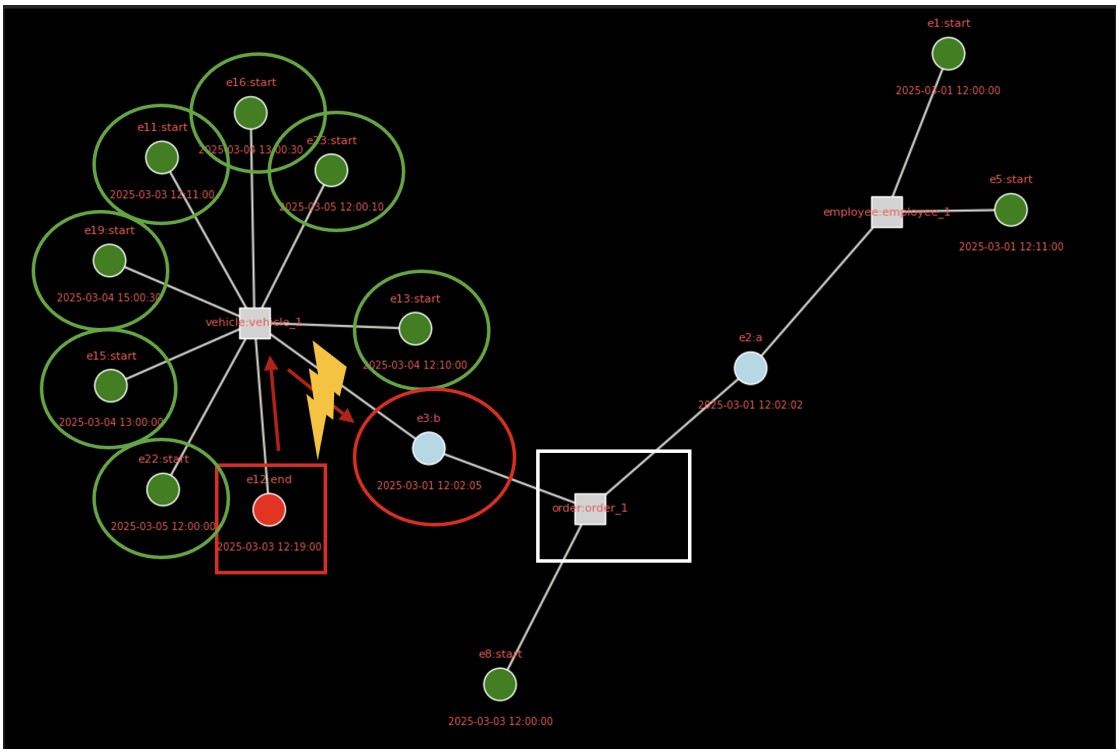


Figure 5.1: Example constellation *(end, start, order)* in the bus transportation event log. Here we see the case when the

Container Logistics

Hing Manufacturing

5.3 Threats to Validity

Here you discuss any element of your experiments (usually depending on the setup), e.g., data sets used, parameters assessed, that might have implications for the validity of your results. For example, if you have not considered a specific range of parameter values, it is unclear how the algorithm will behave for these settings. If you have excluded certain data, this might have its implications for the generalizability of your results. In a way, this is the discussion section of your thesis.

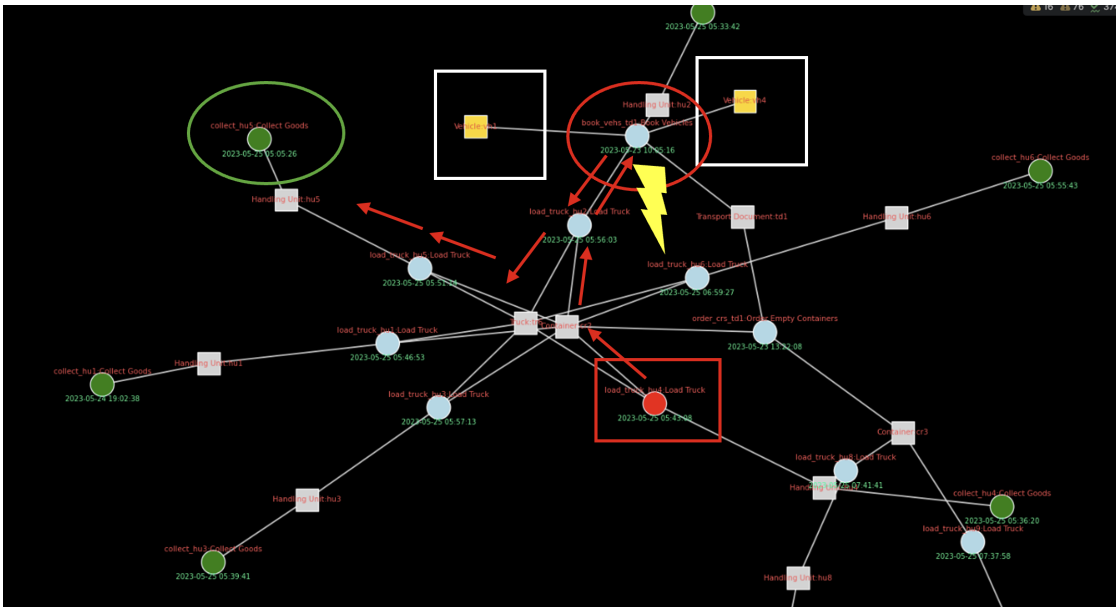


Figure 5.2: Similar to the bus transportation example, both the load truck and both vehicle objects are heading to the collect Goods. But before the vehicle object has to be traversed but due to the time constraint not reachable. In terms of time the reference object lies in front of the the start and end event

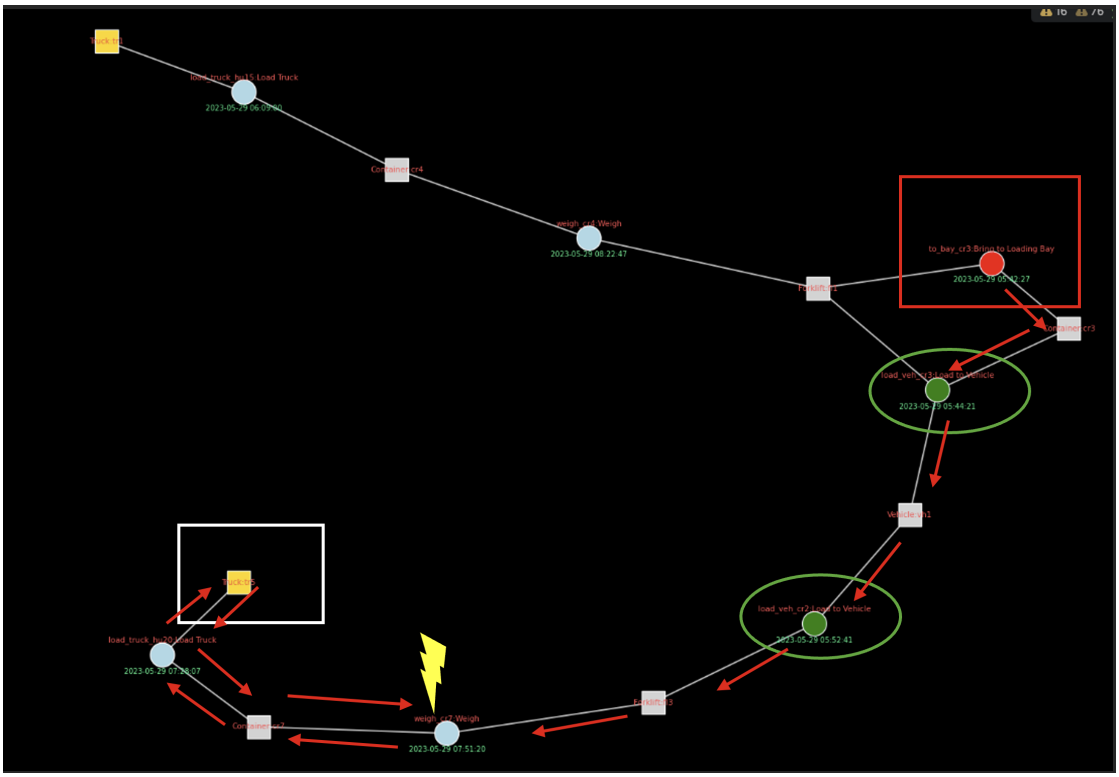


Figure 5.3: Another situation. In terms of time the reference object lies behind the start and end event and is blocked after returning to the end event.

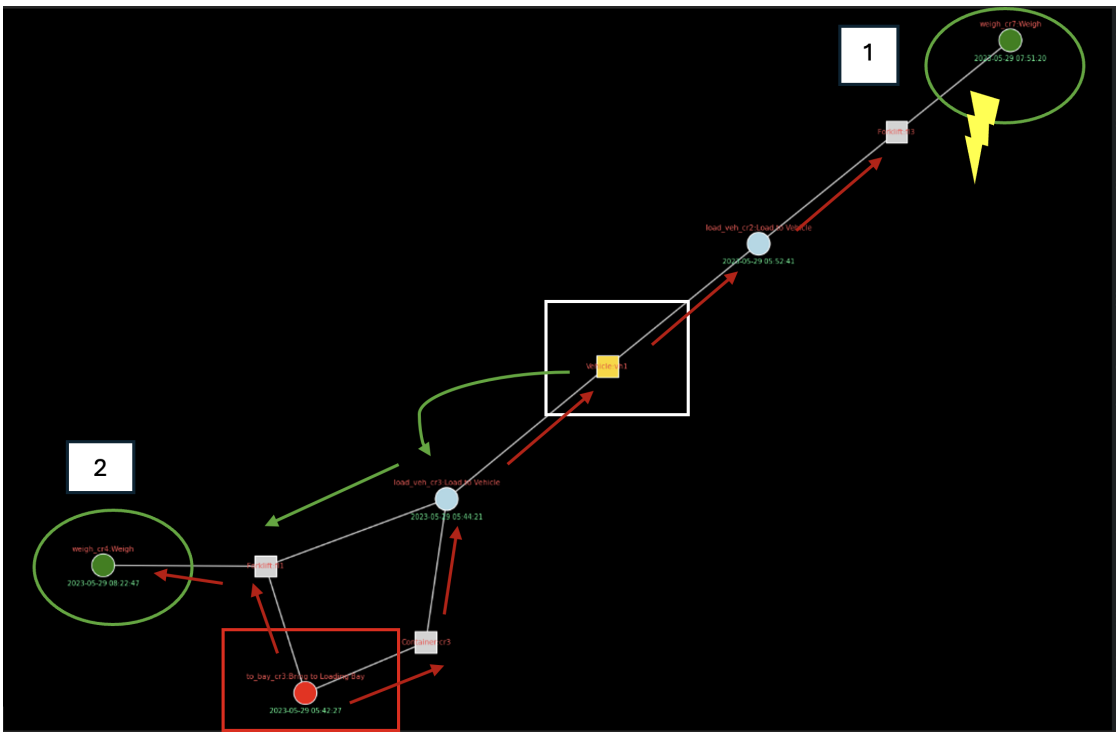


Figure 5.4: The first is that for the first end event and the start event from a time perspective it is good, but it is blocked by the time constraint on the path. After vehicle the timestamps event is bigger than the timestamps end event. Fortunately, the second end event is matched because we allow to after we visited the reference object to turn back.

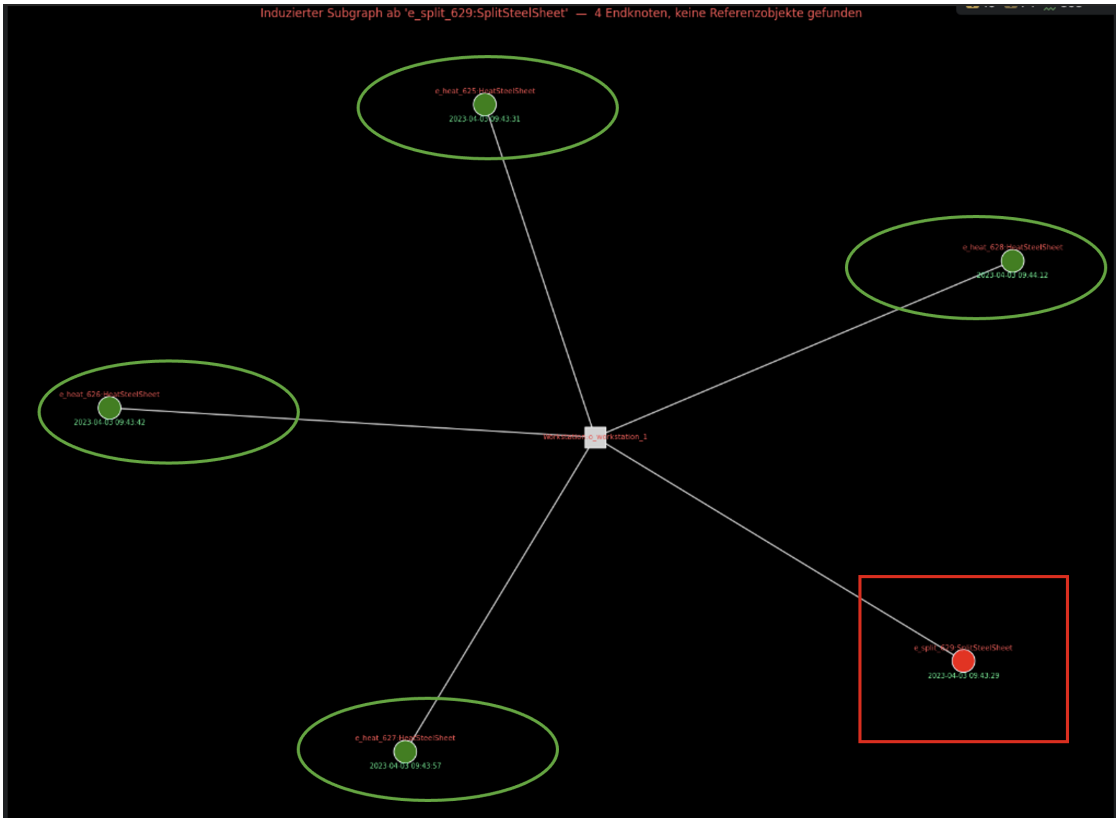


Figure 5.5: the reference objects are out of scope. Empfehlung Tao Object zu machen ähnlich zu Tao activität aus conformance checking

6 Discussion

In this section, you discuss your general approach. What design decisions did you make, and, what are the implications of this? Maybe your approach computes an exact intermediary result, which is costly in terms of time. Does a suitable approximation approach exist? What are the implications for that approach. You can also dive deeper in very related work and explain why it does not work in certain cases.

7 Conclusion

Your conclusion actually follows the same structure as your abstract. However, you present what the reader has seen, again, using the same concepts as you have already used in your abstract.

Future Work. Furthermore, you present interesting directions for future work!

A Some additional stuff

Try to avoid appendices. If you can't, this is a place to show all kinds of results that are supportive, yet, not critical for your thesis.